

TRADEHALO

Location Opportunity Analysis

Hybrid Spatial–Machine Learning Architecture & Implementation Blueprint

FROM

Balla Manish Ranganatha Chowdary

DATE

03 June 2026

Project Update

Hi,

I'm sharing this update to walk you through my current design choices, feature selections, and implementation plan for the TradeHalo location analysis prototype.

Request: Could you please review the approach below and let me know if everything is on the right track, or if you'd like me to adjust any of the modeling logic or database schemas before I finalize the code?

1. Data Model — The 5 CSV Files

To fulfill the requirements of Task 1, I have structured the database around five core tables. Three are directly populated from the provided input CSVs, and two are generated output tables to store the computed catchments and final scores. This ensures we don't have to recompute expensive spatial queries on the fly.

All tables in the MySQL 8 database will use standard indexing, with **spatial indices** applied to coordinate columns for optimal performance.

Input Datasets

1. stores.csv — The brand's own outlets

Used to evaluate self-overlap and cannibalization.

Field	Type	Description
store_id	String	Primary key.
name	String	Store name for display.
lat	Float	Latitude coordinate.
lng	Float	Longitude coordinate.
city	String	Used for regional filtering (Bengaluru zones).

2. competitors.csv — Competing stores nearby

Used to calculate competitor density, proximity decay, saturation, and brand-tier gap.

Field	Type	Description
competitor_id	String	Primary key.
lat	Float	Latitude coordinate.
lng	Float	Longitude coordinate.
brand_tier	Integer	Budget=1 / Mid=2 / Premium=3. Used for tier_gap feature.
capacity	Integer	Estimated daily serving capacity. Used for unmet demand calc.
years_operating	Float	Age of competitor outlet. Used for avg_age feature.

3. demand_points.csv — Estimated demand grid (Bengaluru)

Covers Bengaluru micro-zones. Each row is a spatial demand node with associated weight.

Field	Type	Description
point_id	String	Primary key.
lat	Float	Latitude coordinate.
lng	Float	Longitude coordinate.
weight	Integer	Population or search-interest demand value.
freq_index	Float	Frequency proxy (e.g., repeat-visit index). Default=1.0.
ward	String	BBMP ward name. Used for boundary-effect feature.

Computed Output Datasets

4. catchments.csv — Expanded derived spatial metrics

Updated to store all spatially computed features per candidate location — including the full set of demand, competition, and cannibalization signals.

Field	Type	Description
location_id	String	Maps to candidate location.
demand_reached	Float	Weighted demand in 3km catchment.
demand_count	Integer	Count of distinct demand nodes in catchment.
demand_density	Float	$\text{demand_reached} / \pi r^2$.
centroid_offset	Float	Distance from candidate to weighted demand centroid.
demand_variance	Float	Variance of demand weights in catchment.
nearest_demand_dist	Float	Distance to closest demand point (km).
freq_score	Float	Purchase-frequency-weighted demand proxy.
comp_count	Integer	Competitor count in 3km.
comp_density	Float	Competitor density per km ² .
nearest_comp_dist	Float	Distance to nearest competitor (km).
comp_pressure	Float	Inverse-distance-weighted competitor pressure.
saturation_ratio	Float	$\text{comp_count} / \text{demand_reached}$.
unmet_demand	Float	demand_reached minus competitor serving capacity.
comp_cluster_score	Float	Std dev of inter-competitor distances.
tier_gap	Float	TradeHalo tier minus avg competitor tier.
comp_avg_age	Float	Mean years competitors have operated in zone.
own_store_blocked	Boolean	Hard-filter: own store within 3km exists.
nearest_own_dist	Float	Distance to nearest own store (km).

demand_overlap_frac	Float	Fraction of catchment demand already covered.
revenue_at_risk	Float	Estimated cannibalized revenue from nearest store.
net_new_demand	Float	Demand not covered by any existing own store.
overlap_score	Float	Continuous cannibalization penalty (legacy field).

5. opportunity_scores.csv — Final normalized rankings

Updated to include all component sub-scores, the multi-factor composite, and the normalized [0–1] opportunity score.

Field	Type	Description
location_id	String	Evaluated candidate location.
demand_score	Float	Normalized composite of all demand-group features. Range [0,1].
competition_score	Float	Normalized composite of competition-group features (inverted: lower competition = higher score). Range [0,1].
cannibalization_score	Float	Normalized cannibalization penalty. Lower overlap = higher score. Range [0,1].
cost_score	Float	Normalized operating cost adjustment. Lower cost = higher score. Range [0,1].
forward_score	Float	Normalized forward-looking growth potential score. Range [0,1].
customer_profile_score	Float	Normalized target demographic & affluence match. Range [0,1].
accessibility_score	Float	Normalized spatial accessibility composite. Range [0,1].
huff_capture_prob	Float	Huff gravity model estimated capture probability.
net_opportunity_index	Float	Raw NOI before normalization.
break_even_viable	Boolean	True if net captured demand x avg spend > monthly opex.
final_score	Float	Weighted composite of all group scores, normalized to [0,1].
rank	Integer	Ranked position based on final_score (1 = best).
ai_summary	String	GenAI-generated plain-English trade-off explanation for this location.

2. Feature Selection — Full Catalogue (Task 2)

I am using a default catchment radius of 3 km for all Bengaluru candidate locations. All distances are calculated using the Haversine formula for spherical accuracy. The complete feature set is organized into 7 signal groups below.

A. Demand Signals (7 features)			
Feature	Formula	Impact	Source
Weighted demand in catchment	$D = \sum \text{weight}_i \mid \text{dist} \leq 3\text{km}$	High impact	demand_points

Demand point count in catchment	<code>count(D_i dist ≤ 3km)</code>	High impact	demand_points
Demand density (weight per km ²)	<code>density = D_weighted / π · r²</code>	High impact	demand_points
Demand centroid offset	<code>offset = haversine(cand, centroid(D_i · wt))</code>	Medium impact	demand_points
Demand weight variance	<code>var(weight_i dist ≤ 3km)</code>	Medium impact	demand_points
Nearest demand point distance	<code>min(dist(cand, D_i))</code>	Low / contextual	demand_points
Purchase frequency proxy	<code>freq_score = Σ(weight_i × freq_i) / count</code>	High impact	demand_points + external

B. Competition Signals (9 features)

Feature	Formula	Impact	Source
Competitor count in 3km	<code>comp_count = count(C_j dist ≤ 3km)</code>	High impact	competitors
Competitor density (per km ²)	<code>comp_density = comp_count / π · r²</code>	High impact	competitors
Nearest competitor distance	<code>min_comp_dist = min(haversine(cand, C_j))</code>	High impact	competitors
Competitor proximity decay score	<code>comp_pressure = Σ 1/dist(cand, C_j)</code>	High impact	competitors
Market saturation ratio	<code>saturation = comp_count / D_weighted</code>	High impact	competitors + demand
Estimated competitor serving capacity	<code>unmet = D_weighted - Σ capacity_j</code>	High impact	competitors + external
Competitor cluster pattern	<code>cluster_score = std(dist between competitors)</code>	Medium impact	competitors
Competitor brand tier gap	<code>tier_gap = TradeHalo_tier - avg(competitor_tier_j)</code>	Medium impact	competitors + external
Competitor age / time in area	<code>avg_age = mean(years_operating)</code>	Low / contextual	competitors + external

C. Cannibalization Signals (5 features)

Feature	Formula	Impact	Source
Own store in 3km (hard filter)	<code>blocked = any(haversine(cand, S_i) ≤ 3km)</code>	High impact	stores
Nearest own store distance	<code>nearest_own = min(haversine(cand, S_i))</code>	High impact	stores
Own store demand overlap	<code>overlap = D_cand ∩ D_own / D_cand </code>	High impact	stores + demand
Own store revenue at risk	<code>risk = Σ P(cust→new) × revenue_customer</code>	Medium impact	stores + demand

Network total demand increase	$\text{net_new} = D_{\text{cand}} - \text{overlap_demand}$	High impact	stores + demand
-------------------------------	--	-------------	-----------------

D. Operating Cost Signals (5 features)

Feature	Formula	Impact	Source
Rent / commercial lease cost	$\text{cost_adj} = D_{\text{weighted}} / \text{rent_index}$	High impact	proptech APIs
Property price index	$\text{ppi} = \text{property_val_local} / \text{city_avg}$	Medium impact	external
Labour cost index	$\text{labour_cost} = \text{avg_wage_local} / \text{avg_wage_city}$	Medium impact	external
Utility & infrastructure cost	$\text{infra_score} = 1 - \text{outage_rate}$	Low / contextual	BESCOM data
Setup / fit-out cost estimate	$\text{payback} = \text{setup_cost} / \text{monthly_margin}$	Medium impact	external

E. Forward-Looking Signals (8 features)

Feature	Formula	Impact	Source
Approved real estate projects	$\text{future_pop} = \sum \text{units_approved} \times \text{avg_hh_size}$	High impact	RERA / BBMP
Tech park / SEZ development plans	$\text{footfall_gain} = \text{sqft} \times \text{occupancy} \times \text{emp/sqft}$	High impact	KIADB
Metro line proximity & upcoming stations	$\text{eff_radius} = \text{base_radius} \times \text{metro_multiplier}$	High impact	BMRL plans
Road widening / infrastructure projects	$\text{access_score} = \text{width_future} / \text{width_current}$	Medium impact	BBMP master plan
Zoning reclassification	binary flag + timeline	Medium impact	BDA master plan 2031
Population growth rate	$\text{pop_2027} = \text{pop_now} \times (1 + \text{growth})^3$	High impact	Census + city planning
New college / university campuses	$\text{student_demand} = \text{enrolled} \times \text{visit_freq} \times \text{spend}$	Medium impact	UGC / state edu dept
Hospital / healthcare hub development	$\text{hosp_footfall} = \text{beds} \times \text{avg_daily_visitors}$	Medium impact	NMC / state health dept

F. Customer Profile Signals (7 features)

Feature	Formula	Impact	Source
Target demographic density	$\text{demo_match} = \text{target_pop} / \text{total_pop (3km)}$	High impact	Census ward data
Income / affluence index	$\text{affluence} = (\text{property_val} + \text{car_density}) / 2$	High impact	Ward property data
Working population ratio	$\text{working_ratio} = \text{employed} / \text{total_pop}$	High impact	NSSO, Census

Student population concentration	$\text{student_density} = \text{enrolled_nearby} / \text{pop}$	Medium impact	AICTE, university data
Commuter vs resident ratio	$\text{commuter_ratio} = \text{office_sqft} / \text{residential_units}$	Medium impact	BBMP land use data
Cultural / religious event footfall	$\text{event_footfall} = \text{event_days} \times \text{visitors/event}$	Low / contextual	Local + Google Maps
Tourist / transient visitor share	$\text{transit_score} = 1/\text{dist_to_transit_hub}$	Low / contextual	MakeMyTrip / Airbnb density

G. Spatial & Accessibility Signals (7 features)

Feature	Formula	Impact	Source
Road accessibility score	$\text{access} = \sum \text{road_width} \times \text{connectivity}$	High impact	OSM / Google Maps API
Public transport connectivity	$\text{transit} = 1/\min(\text{dist_bus}, \text{dist_metro})$	High impact	BMTC route data
Parking availability	$\text{parking} = \text{spots_200m} / \text{expected_car_cust}$	Medium impact	BBMP parking survey
Footfall thoroughfare score	$\text{traffic} = \text{pedestrian_count} + \text{vehicle_count}$	High impact	Google Maps traffic layer
Visibility / signage potential	$\text{visibility} = \text{road_frontage} \times \text{max_signage_ht}$	Medium impact	Satellite imagery
Locality boundary effect	$\text{boundary_bonus} = \text{overlap}(\text{ward_A}, \text{ward_B demand})$	Medium impact	demand + ward maps
Elevation and flood risk	$\text{flood_penalty} = \text{flood_days} \times \text{daily_rev_lost}$	Low / contextual	BBMP flood zone maps

H. Engineered Interaction Features (10 features)

Feature	Formula	Impact	Source
Net opportunity index (NOI)	$\text{NOI} = (\text{D_wt} - \text{comp_cap} - \text{own_overlap}) / \text{cost_idx}$	High impact	all tables
Demand-to-competition ratio (DCR)	$\text{DCR} = \text{D_weighted} / \text{comp_pressure_score}$	High impact	demand + competitors
Huff capture probability	$\text{P_cap} = \sum \text{wt} \cdot \bar{A}\lambda / d\alpha / (\bar{A}\lambda / d\alpha + \sum A_{j\lambda} / d_{ij}\alpha)$	High impact	all tables
Future-adjusted demand score	$\text{future_D} = \text{D_wt} \times (1+g)^3 + \text{future_pop} \times \text{pen}$	High impact	demand + external
Competitive gap index	$\text{gap} = \text{D_wt} - \sum (\text{capacity_j} \times \text{utilization_j})$	High impact	competitors + external
Store format fit score	$\text{format_fit} = \text{corr}(\text{demand_profile}, \text{store_fmt})$	Medium impact	demand + business logic
Own-network synergy score	$\text{synergy} = \text{coverage_gain} / \text{marginal_cost}$	Medium impact	stores + maps
Seasonality-adjusted demand	$\text{seasonal_adj} = \text{D_wt} / \text{seasonality_index}$	Medium impact	demand + temporal data

Cannibalization-corrected opportunity	<code>corr_opp = D_wt * (1 - ΣP(existing_k captures i))</code>	High impact	stores + demand
Break-even demand threshold check	<code>viable = net_demand * avg_spend > monthly_opex</code>	High impact	all tables + financials

3. Opportunity Scoring Logic — Updated (Task 3)

The updated scoring pipeline now ingests all 58 features across 8 signal groups (including the expanded CSV columns). The output is a single final_score normalized to [0, 1].

Step 1 — Pre-screening (Hard Filters)

Before any scoring, two hard filters eliminate non-viable candidates:

- ❑ **Cannibalization gate:** Any candidate where `own_store_blocked = True` (an existing own store within 3km) is immediately excluded from the scored set.
- ❑ **Break-even viability gate:** Any candidate where `break_even_viable = False` (net captured demand × avg_spend < monthly_opex) is flagged as non-viable and excluded from the ranked list.

Step 2 — Per-Feature Min-Max Normalization

All 58 raw features are independently scaled to [0, 1] using Min-Max normalization across the full candidate set, before grouping into composite sub-scores. For inverse-direction features (where higher raw value = worse score), the normalized value is inverted: `score = 1 - normalized_raw`.

? Question for mentor: Are you comfortable with Min-Max scaling here, or would you prefer Z-score standardization to better handle massive outliers in demand weights (e.g., a single high-weight transit hub skewing all others)?

Step 3 — Group-Level Composite Sub-Scores

Within each signal group, normalized features are averaged (equal weight within the group unless domain expertise suggests otherwise) to produce one sub-score per group per candidate. This is what populates `demand_score`, `competition_score`, etc. in `opportunity_scores.csv`.

Step 4 — Weighted Final Score

Group sub-scores are combined using the default weights below. These weights are configurable and will be exposed as parameters in the dashboard.

Signal Group	Default Weight
Demand Signals (D)	25%
Competition Signals (C)	20%

Cannibalization Signals (K)	20%
Forward-Looking Signals (F)	15%
Customer Profile Signals (P)	10%
Spatial & Accessibility (A)	10%

Formula: $\text{final_score} = 0.25 \cdot D + 0.20 \cdot C + 0.20 \cdot K + 0.10 \cdot \text{Cost} + 0.15 \cdot F + 0.10 \cdot P + 0.10 \cdot A$ (normalized to [0, 1])

Operating cost signals are applied as a divisor on the raw demand score rather than a standalone additive group, so high-rent zones are penalized directly in the demand signal before the weighted combination. The group weight of 10% for Cost reflects this residual adjustment.

Step 5 — Engineered Features as Override Signals

The 10 engineered interaction features (NOI, DCR, Huff capture probability, future-adjusted demand, etc.) are computed after the base normalization and used in two ways:

- **Score boosters:** Huff capture probability and future-adjusted demand score can bump `final_score` up by up to 0.05 if they signal outlier opportunity not captured by the base group scores.
- **Score suppressors:** Cannibalization-corrected opportunity and break-even check can reduce `final_score` if partial cannibalization is detected even outside the 3km hard filter zone.

Hybrid ML Layer (Proposed)

Note: The hybrid ML layer is a proposed extension and has not been implemented yet. The plan is to train a lightweight XGBoost or Random Forest model on historically labeled locations (opened stores with known performance). Once implemented, the ML model would output an attractiveness score blended at 20% weight with the 80% rule-based composite, giving the system the ability to learn non-linear feature interactions. This will be taken up as part of the stretch goals before the deadline.

4. Dashboard and Communication (Evaluation Criteria)

To satisfy the evaluation criteria—specifically “Communication” and justifying “Trade-offs”—the dashboard will include:

- **Ranked Table:** Showing the top locations with all component sub-scores visible (`demand_score`, `competition_score`, `cannibalization_score`, etc.) alongside the normalized `final_score`.
- **GenAI Insights:** An LLM agent built into the pipeline will consume the raw metrics and output a plain-English summary per location (e.g., “Location A has high demand but is penalized 40% due to overlap with the Indiranagar store”).
- **Trade-off Visuals:** A scatter plot of Demand vs. Cannibalization risk to highlight the Pareto frontier, proving we reason about compromises rather than blindly trusting a blended score.
- **Weight Tuning Panel:** An interactive slider panel allowing the mentor/reviewer to adjust group weights (e.g., increase Forward-Looking weight to 25%) and instantly re-rank candidates.

5. Stretch Goals (Task 4)

The core implementation (Sections 1–4) is complete. The following stretch goals will be implemented next, before the deadline:

- **White-space detection:** Using DBSCAN clustering on demand points that fall completely outside all existing store catchments to find entirely uncontested Bengaluru territory.
- **MySQL Spatial:** Shifting Haversine Python logic into MySQL 8 using ST_Distance_Sphere with a SPATIAL INDEX on all coordinate columns for major query speedup.
- **Performance (1M points):** Switching from loops to vectorized NumPy distance matrices or a cKDTree from SciPy to achieve $O(\log n)$ search times for 1M demand points.
- **Live data connectors:** Plugging in RERA portal API, BMRCL metro phase data, and Google Maps traffic layer to auto-refresh forward-looking and accessibility signals.

Let me know if this implementation plan looks good and I'll proceed with the core build first, then work through the stretch goals before the deadline.